Learning of Auto-regressive Models

Tirtharaj Dash

Department of CS & IS
BITS Pilani, Goa Campus, India
Homepage

April 2026

Recap: the AR model

We want to model $P(w_1, w_2, \ldots, w_T)$ via the chain rule:

$$P(w_1, \ldots, w_T) = \prod_{t=1}^{T} P(w_t \mid w_{<t})$$

We can use *count-based* estimates $P(\cdot|\cdot)$:

$$\hat{P}(w_t \mid w_{t-1}) = \frac{C(w_{t-1}, w_t)}{C(w_{t-1})}$$

Problems: data sparsity, no generalisation, fixed context window.

Recap: the AR model

Question: How do we *learn* a parametric function

$$f_\theta : (w_1, \ldots, w_{t-1}) \;\mapsto\; P_\theta(\cdot \mid w_{<t})$$

from data, using gradient-based optimisation?

The training objective

Given a corpus $\mathcal{D} = \{w_1, w_2, \ldots, w_T\}$, we maximise the log-likelihood:

$$\max_{\theta} \sum_{t=1}^{T} \log P_\theta(w_t \mid w_{<t})$$

equivalently, minimise the **negative log-likelihood (NLL)**:

$$\mathcal{L}(\theta) = -\frac{1}{T} \sum_{t=1}^{T} \log P_\theta(w_t \mid w_{<t})$$

This is identical to the **cross-entropy** between the model distribution and the one-hot data distribution at each position t.

The training objective

Connection to perplexity (PPL):

$$\mathrm{PPL} = e^{\mathcal{L}(\theta)}$$

$$\boxed{\text{Minimising NLL} \Leftrightarrow \text{minimising PPL} \Leftrightarrow \text{maximising data log-likelihood.}}$$

The training objective

At each position t, the model outputs a probability vector over the vocabulary $\mathcal{V}$ of size V:

$$\hat{\boldsymbol{y}}_t = P_\theta(\cdot \mid w_{<t}) \in \mathbb{R}^V, \quad \sum_{v=1}^{V} \hat{y}_{t,v} = 1$$

The true target is a one-hot vector $\boldsymbol{y}_t \in \{0,1\}^V$ with $y_{t,w_t} = 1$.

The training objective

Per-token cross-entropy loss:

$$\ell_t = -\sum_{v=1}^{V} y_{t,v} \log \hat{y}_{t,v} = -\log \hat{y}_{t,w_t}$$

Total loss (averaged over T positions):

$$\mathcal{L}(\theta) = \frac{1}{T} \sum_{t=1}^{T} \ell_t$$

We need $\nabla_\theta \mathcal{L}(\theta)$ to update the parameters via gradient descent.

Background: backpropagation

Refer to: Lecture 3 and Lecture 4 of CS F425 (my *old* DL course).

Backpropagation applies the **chain rule** of calculus to propagate gradients from the loss $\mathcal{L}$ backwards through a computational graph.

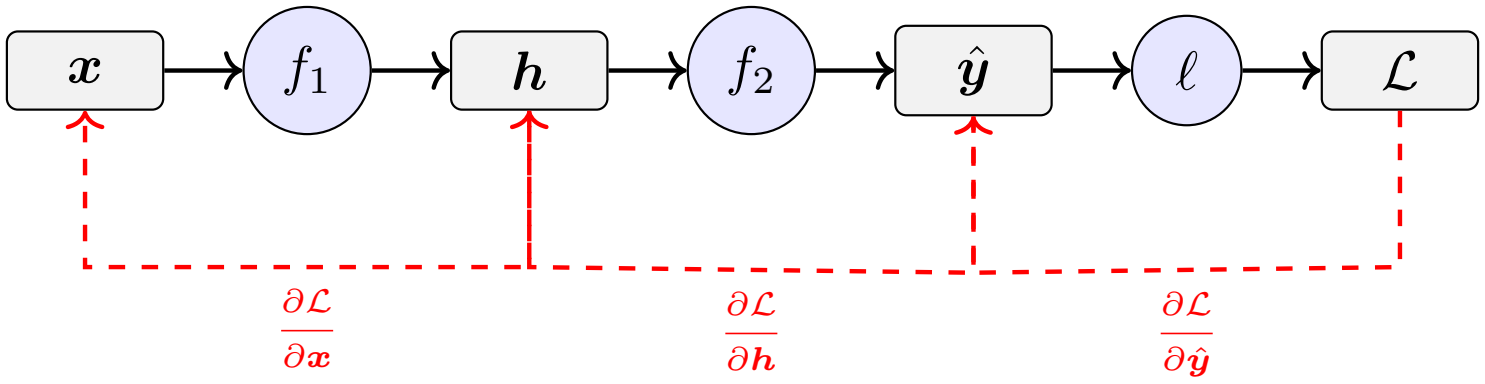
For a composition $\mathcal{L} = g(f(\boldsymbol{x}))$:

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{x}} = \frac{\partial \mathcal{L}}{\partial f} \cdot \frac{\partial f}{\partial \boldsymbol{x}}$$

Key insight: every differentiable operation in a neural network can be decomposed into primitive operations whose local Jacobians are easy to compute.

Background: backpropagation

Computational graph view:



Each trainable parameter θ_i lies along one of these paths. Its gradient is accumulated as the product of local Jacobians along the path from $\mathcal{L}$ to θ_i.

Example

We use a tiny AR model to trace the chain rule step by step.

Setting:

- Vocabulary: $V = 3$ tokens — `the` (0), `cat` (1), `sat` (2)
- Embedding dim: $d = 2$; hidden dim: $d_h = 2$ (one hidden layer, $\tanh$)
- Output: $V = 3$ (softmax over vocabulary)

Example

Forward graph:

$$\boldsymbol{e} \xrightarrow{\boldsymbol{W}_h, \boldsymbol{b}_h} \boldsymbol{a}_h \xrightarrow{\tanh} \boldsymbol{h} \xrightarrow{\boldsymbol{W}_o, \boldsymbol{b}_o} \boldsymbol{z} \xrightarrow{\text{softmax}} \hat{\boldsymbol{y}} \xrightarrow{\text{CE}} \ell$$

Parameters (initialised):

$$\boldsymbol{W}_h = \begin{bmatrix} 0.5 & 0.2 \\ -0.3 & 0.8 \end{bmatrix}, \ \boldsymbol{b}_h = \begin{bmatrix} 0.1 \\ -0.1 \end{bmatrix}, \ \boldsymbol{W}_o = \begin{bmatrix} 1.0 & 0.0 \\ 0.0 & 1.0 \\ -1.0 & 0.0 \end{bmatrix}, \ \boldsymbol{b}_o = \boldsymbol{0}$$

Input / target: Predict next token after `the`: embedding $\boldsymbol{e} = [1.0,\ 0.5]^\mathsf{T}$, target $\boldsymbol{y} = [0,\ 1,\ 0]^\mathsf{T}$ (i.e., `cat`).

Example

Forward pass:

Step F1. Pre-activation of hidden layer:

$$\boldsymbol{a}_h = \boldsymbol{W}_h \boldsymbol{e} + \boldsymbol{b}_h = \begin{bmatrix} 0.5 \times 1.0 + 0.2 \times 0.5 \\ -0.3 \times 1.0 + 0.8 \times 0.5 \end{bmatrix} + \begin{bmatrix} 0.1 \\ -0.1 \end{bmatrix} = \begin{bmatrix} 0.7 \\ 0.0 \end{bmatrix}$$

Step F2. Hidden activation ($\tanh$):

$$\boldsymbol{h} = \tanh(\boldsymbol{a}_h) = \begin{bmatrix} \tanh(0.7) \\ \tanh(0.0) \end{bmatrix} \approx \begin{bmatrix} 0.604 \\ 0.000 \end{bmatrix}$$

Example

Step F3. Output logits:

$$\boldsymbol{z} = \boldsymbol{W}_o \boldsymbol{h} + \boldsymbol{b}_o = \begin{bmatrix} 1.0 \times 0.604 \\ 0.0 \times 0.604 \\ -1.0 \times 0.604 \end{bmatrix} = \begin{bmatrix} 0.604 \\ 0.000 \\ -0.604 \end{bmatrix}$$

Step F4. Softmax:

$$e^{0.604} \approx 1.829, \quad e^{0.000} = 1.000, \quad e^{-0.604} \approx 0.547, \quad \sum = 3.376$$

$$\hat{\boldsymbol{y}} = \begin{bmatrix} 1.829/3.376 \\ 1.000/3.376 \\ 0.547/3.376 \end{bmatrix} \approx \begin{bmatrix} 0.542 \\ 0.296 \\ 0.162 \end{bmatrix}$$

Example

Step F5. Cross-entropy loss (target: class $1 = \texttt{cat}$):

$$\ell = -\log \hat{y}_1 = -\log(0.296) \approx 1.216$$

So the model currently assigns probability 0.296 to the correct token `cat`; we need backprop to push this higher.

Example

We now apply the chain rule layer by layer, flowing gradients backward.

Step B1. Gradient at the output logits (softmax + cross-entropy combined):

$$\frac{\partial \ell}{\partial \boldsymbol{z}} = \hat{\boldsymbol{y}} - \boldsymbol{y} = \begin{bmatrix} 0.542 \\ 0.296 \\ 0.162 \end{bmatrix} - \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0.542 \\ -0.704 \\ 0.162 \end{bmatrix}$$

Interpretation: the model over-predicted `the` (+0.542) and `sat` (+0.162), and under-predicted `cat` (-0.704). Gradients will correct this.

Example

Step B2. Gradient w.r.t. $\boldsymbol{W}_o$ (outer product rule, $\boldsymbol{z} = \boldsymbol{W}_o \boldsymbol{h}$):

$$
\begin{aligned}
\frac{\partial \ell}{\partial \boldsymbol{W}_o} &= \frac{\partial \ell}{\partial \boldsymbol{z}} \otimes \boldsymbol{h}^\mathsf{T} \\
&= \begin{bmatrix} 0.542 \\ -0.704 \\ 0.162 \end{bmatrix} \begin{bmatrix} 0.604 & 0.000 \end{bmatrix} \\
&= \begin{bmatrix} 0.327 & 0.000 \\ -0.425 & 0.000 \\ 0.098 & 0.000 \end{bmatrix}
\end{aligned}
$$

Example

Step B3. Gradient flowing into hidden layer:

$$
\begin{aligned}
\frac{\partial \ell}{\partial \boldsymbol{h}} &= \boldsymbol{W}_o^\top \frac{\partial \ell}{\partial \boldsymbol{z}} \\
&= \begin{bmatrix} 1.0 & 0.0 & -1.0 \\ 0.0 & 1.0 & 0.0 \end{bmatrix} \begin{bmatrix} 0.542 \\ -0.704 \\ 0.162 \end{bmatrix} \\
&= \begin{bmatrix} 0.542 - 0.162 \\ -0.704 \end{bmatrix} \\
&= \begin{bmatrix} 0.380 \\ -0.704 \end{bmatrix}
\end{aligned}
$$

Example

(HL) Step B4. Gradient through $\tanh$ (element-wise; $\tanh' = 1 - \tanh^2$):

$$\frac{\partial \ell}{\partial \boldsymbol{a}_h} = \frac{\partial \ell}{\partial \boldsymbol{h}} \odot (1 - \boldsymbol{h}^2)$$

$$\boldsymbol{h}^2 = \begin{bmatrix} 0.604^2 \\ 0.000^2 \end{bmatrix} = \begin{bmatrix} 0.365 \\ 0.000 \end{bmatrix}, \qquad 1 - \boldsymbol{h}^2 = \begin{bmatrix} 0.635 \\ 1.000 \end{bmatrix}$$

$$\frac{\partial \ell}{\partial \boldsymbol{a}_h} = \begin{bmatrix} 0.380 \times 0.635 \\ -0.704 \times 1.000 \end{bmatrix} = \begin{bmatrix} 0.241 \\ -0.704 \end{bmatrix}$$

Do you notice any issue?

Example

Note: when $|h_i|$ is close to 1, $1 - h_i^2 \approx 0$, i.e., the $\tanh$ is saturated and the gradient is almost zero — the root cause of vanishing gradients in deep/recurrent networks.

Example

Step B5. Gradient w.r.t. $\boldsymbol{W}_h$ (outer product):

$$\frac{\partial \ell}{\partial \boldsymbol{W}_h} = \frac{\partial \ell}{\partial \boldsymbol{a}_h} \otimes \boldsymbol{e}^\mathsf{T} = \begin{bmatrix} 0.241 \\ -0.704 \end{bmatrix} \begin{bmatrix} 1.0 & 0.5 \end{bmatrix} = \begin{bmatrix} 0.241 & 0.121 \\ -0.704 & -0.352 \end{bmatrix}$$

Step B6. Gradient w.r.t. $\boldsymbol{b}_h$:

$$\frac{\partial \ell}{\partial \boldsymbol{b}_h} = \frac{\partial \ell}{\partial \boldsymbol{a}_h} = \begin{bmatrix} 0.241 \\ -0.704 \end{bmatrix}$$

Example

Step B7. Gradient into the embedding e (gradient of the input, propagates further back to $\boldsymbol{E}$):

$$
\begin{aligned}
\frac{\partial \ell}{\partial \boldsymbol{e}} &= \boldsymbol{W}_h^{\mathsf{T}} \frac{\partial \ell}{\partial \boldsymbol{a}_h} \\
&= \begin{bmatrix} 0.5 & -0.3 \\ 0.2 & 0.8 \end{bmatrix} \begin{bmatrix} 0.241 \\ -0.704 \end{bmatrix} \\
&= \begin{bmatrix} 0.5 \times 0.241 + (-0.3) \times (-0.704) \\ 0.2 \times 0.241 + 0.8 \times (-0.704) \end{bmatrix} \\
&= \begin{bmatrix} 0.332 \\ -0.515 \end{bmatrix}
\end{aligned}
$$

Example

All computed gradients:

Parameter	Gradient	Update direction
$\boldsymbol{W}_o$	$\begin{bmatrix} 0.327 & 0 \\ -0.425 & 0 \\ 0.098 & 0 \end{bmatrix}$	decrease $\boldsymbol{W}_o$ by $\eta \cdot \nabla$
$\boldsymbol{W}_h$	$\begin{bmatrix} 0.241 & 0.121 \\ -0.704 & -0.352 \end{bmatrix}$	decrease $\boldsymbol{W}_h$ by $\eta \cdot \nabla$
$\boldsymbol{b}_h$	$[0.241,\ -0.704]^\mathsf{T}$	
$\boldsymbol{e}$ (embedding)	$[0.332,\ -0.515]^\mathsf{T}$	update $\boldsymbol{E}[\texttt{the}]$

Example

After one gradient step with learning rate η:

$$\boldsymbol{W}_h \leftarrow \boldsymbol{W}_h - \eta \begin{bmatrix} 0.241 & 0.121 \\ -0.704 & -0.352 \end{bmatrix},$$

$$\boldsymbol{W}_o \leftarrow \boldsymbol{W}_o - \eta \begin{bmatrix} 0.327 & 0 \\ -0.425 & 0 \\ 0.098 & 0 \end{bmatrix}$$

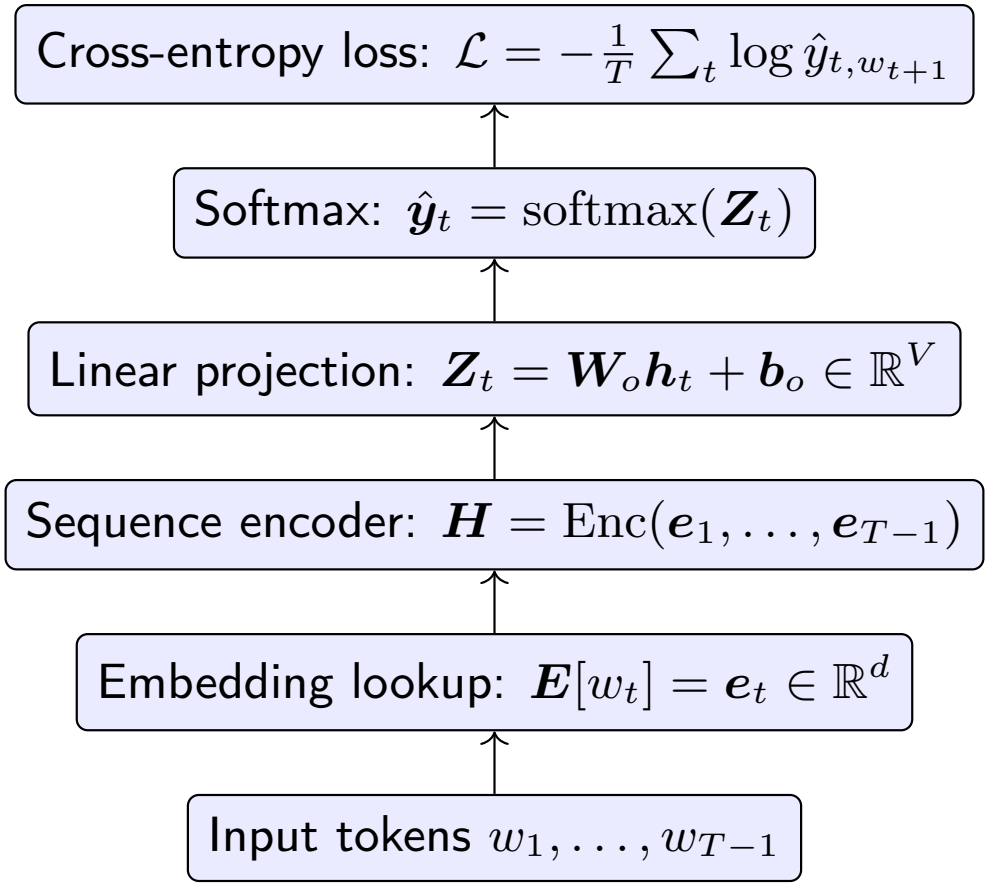
For the embedding: only the row for `the` (index 0) is updated; all other rows remain unchanged. Because the other indices never contributed anything to this loss.

Forward pass of a neural AR model

We describe the forward pass (and later the backward pass) for a generic neural AR model with three components:

1. Embedding lookup
2. Sequence encoder (RNN or Transformer)
3. Output projection + softmax

Forward pass of a neural AR model



Target at position t: the *next* token w_{t+1} (teacher forcing).

Backprop: output layer (softmax + cross-entropy)

The softmax at position t:

$$\hat{y}_{t,j} = \frac{\exp(z_{t,j})}{\sum_{k=1}^{V} \exp(z_{t,k})}$$

Per-token loss: $\ell_t = -\log \hat{y}_{t,w_{t+1}}$ (only the correct class contributes).

Backprop: output layer (softmax + cross-entropy)

Gradient of ℓ_t w.r.t. logits $\boldsymbol{z}_t$ (standard result):

$$\frac{\partial \ell_t}{\partial z_{t,j}} = \hat{y}_{t,j} - y_{t,j}$$

where $y_{t,j} = \mathbf{1}[j = w_{t+1}]$.

In vector form:

$$\nabla_{\boldsymbol{z}_t} \ell_t = \hat{\boldsymbol{y}}_t - \boldsymbol{y}_t$$

This is the gradient entering the encoder from the top.

Backprop: output layer (softmax + cross-entropy)

Gradient w.r.t. output projection parameters:
Given $\boldsymbol{z}_t = \boldsymbol{W}_o \boldsymbol{h}_t + \boldsymbol{b}_o$:

$$\frac{\partial \ell_t}{\partial \boldsymbol{W}_o} = (\hat{\boldsymbol{y}}_t - \boldsymbol{y}_t)\boldsymbol{h}_t^\mathsf{T}, \qquad \frac{\partial \ell_t}{\partial \boldsymbol{b}_o} = \hat{\boldsymbol{y}}_t - \boldsymbol{y}_t$$

Gradient flowing into the encoder (δ_t entering from top):

$$\frac{\partial \ell_t}{\partial \boldsymbol{h}_t} = \boldsymbol{W}_o^\mathsf{T}(\hat{\boldsymbol{y}}_t - \boldsymbol{y}_t)$$

This vector $\boldsymbol{\delta}_t \in \mathbb{R}^d$ is then propagated back through the encoder.

Backprop: output layer (softmax + cross-entropy)

Note: Because the total loss is $\mathcal{L} = \frac{1}{T}\sum_t \ell_t$, gradients from all T positions are accumulated (averaged) when updating shared parameters like $\boldsymbol{W}_o$.

Backprop: embedding lookup

The embedding matrix $\boldsymbol{E} \in \mathbb{R}^{V \times d}$ maps each token w_t to a row vector $\boldsymbol{e}_t = \boldsymbol{E}[w_t]$.

There is no continuous operation here — it is a table lookup.

Gradients are computed as:

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{E}[v]} = \sum_{t:w_t=v} \frac{\partial \mathcal{L}}{\partial \boldsymbol{e}_t}$$

i.e., the gradient for token v's embedding is the *sum* of upstream gradients from every position where token v appears.

Backprop: embedding lookup

Sparsity: for a given sequence, most rows of $\boldsymbol{E}$ receive zero gradient (corresponding to tokens that do not appear). In practice this is handled by *sparse gradient updates* or just by zeroing unused gradients.

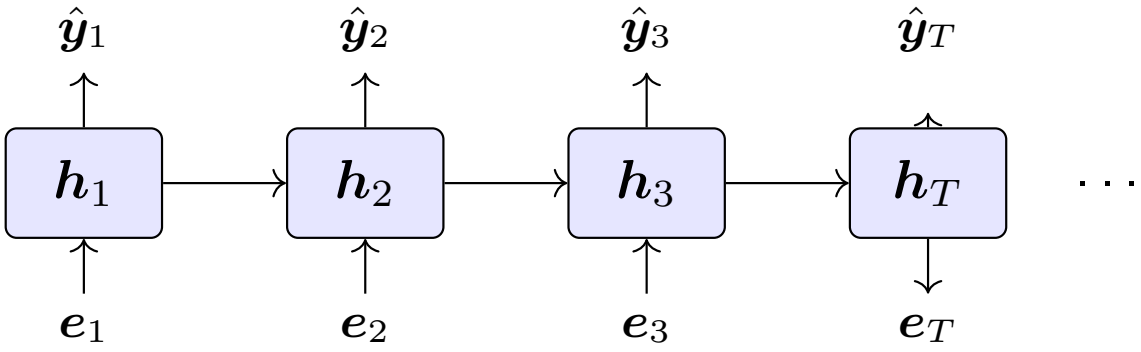
The embedding gradients train the embeddings so that tokens with similar distributional contexts end up with similar representations.

RNN as an AR model

A recurrent AR model processes tokens one by one:

$$\boldsymbol{h}_t = \tanh(\boldsymbol{W}_h \boldsymbol{h}_{t-1} + \boldsymbol{W}_x \boldsymbol{e}_{w_t} + \boldsymbol{b})$$

$$\hat{\boldsymbol{y}}_t = \mathrm{softmax}(\boldsymbol{W}_o \boldsymbol{h}_t)$$



Parameters: $\boldsymbol{W}_h, \boldsymbol{W}_x, \boldsymbol{b}, \boldsymbol{W}_o$ (shared across all time steps).

Backpropagation through time (BPTT)

Refer to: Lecture 18 (BPTT) of CS F425.

Since $\boldsymbol{h}_t$ depends on $\boldsymbol{h}_{t-1}$, the total gradient for $\boldsymbol{W}_h$ sums over all positions:

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{W}_h} = \sum_{t=1}^{T} \frac{\partial \ell_t}{\partial \boldsymbol{W}_h}$$

Each term requires backpropagating through the recurrence. For position t, by the chain rule:

$$\frac{\partial \ell_t}{\partial \boldsymbol{W}_h} = \sum_{k=1}^{t} \frac{\partial \ell_t}{\partial \boldsymbol{h}_t} \left(\prod_{j=k+1}^{t} \frac{\partial \boldsymbol{h}_j}{\partial \boldsymbol{h}_{j-1}} \right) \frac{\partial \boldsymbol{h}_k}{\partial \boldsymbol{W}_h}$$

The product $\prod_{j=k+1}^{t} \frac{\partial \boldsymbol{h}_j}{\partial \boldsymbol{h}_{j-1}}$ is a product of $t-k$ Jacobians.

Backpropagation through time (BPTT)

Local Jacobian: for $\boldsymbol{h}_t = \tanh(\boldsymbol{W}_h \boldsymbol{h}_{t-1} + \boldsymbol{W}_x \boldsymbol{e}_{w_t} + \boldsymbol{b})$:

$$\frac{\partial \boldsymbol{h}_t}{\partial \boldsymbol{h}_{t-1}} = \operatorname{diag}(1 - \boldsymbol{h}_t^2)\, \boldsymbol{W}_h$$

(using $\tanh' = 1 - \tanh^2$).

The product of k such Jacobians over time steps:

$$\prod_{j=t-k+1}^{t} \frac{\partial \boldsymbol{h}_j}{\partial \boldsymbol{h}_{j-1}} = \prod_{j=t-k+1}^{t} \operatorname{diag}(1 - \boldsymbol{h}_j^2)\, \boldsymbol{W}_h$$

When k is large, this product either *vanishes* (if $\|\boldsymbol{W}_h\| < 1$) or *explodes* (if $\|\boldsymbol{W}_h\| > 1$).

Backpropagation through time (BPTT)

Vanishing gradients:

- Gradients for tokens many steps in the past become negligibly small.
- The model cannot effectively learn long-range dependencies.

Exploding gradients:

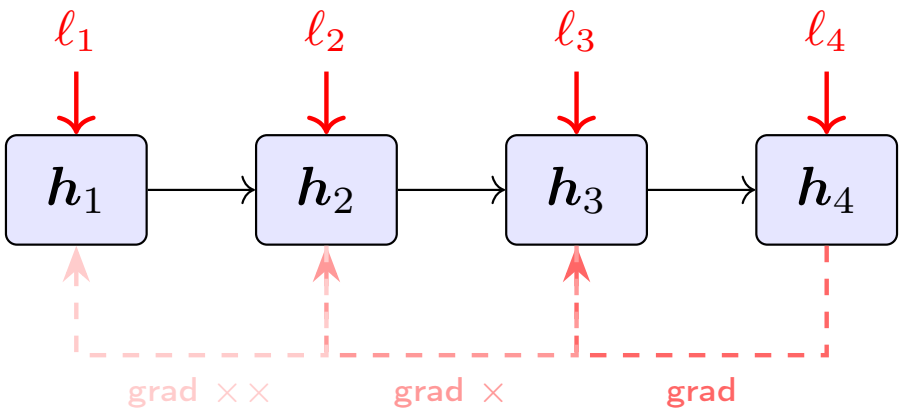
- Norms grow exponentially, destabilising training.
- Mitigation: *gradient clipping* — rescale ∇ if $\|\nabla\| > \tau$.

Refer to: Lecture 19 (vanishing gradients in RNN) of CS F425.

Truncated BPTT: in practice, only unroll for k steps (e.g., $k = 64$ or $k = 128$) to limit computation and mitigate vanishing gradients.

Backpropagation through time (BPTT)

Visualising gradient flow in BPTT:

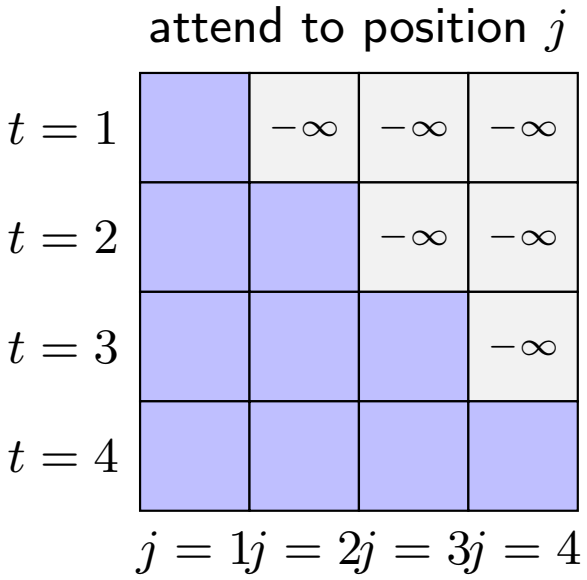


gradient decays with distance

LSTMs (Hochreiter & Schmidhuber, 1997) address vanishing gradients with a gated *cell state* $\boldsymbol{c}_t$ that passes gradients more directly through time, but the fundamental bottleneck remains.

Transformer as an AR model

The Transformer (Vaswani et al., 2017) processes all positions in parallel. It uses **masked (causal) self-attention** to ensure that position t can only attend to positions $\leq t$.



Transformer as an AR model

Masked attention score: $a_{t,j} = \frac{\boldsymbol{q}_t^\top \boldsymbol{k}_j}{\sqrt{d_k}} + M_{t,j}$, where $M_{t,j} = -\infty$ for $j > t$.

After softmax: $\alpha_{t,j} = \mathrm{softmax}(a_{t,\cdot})_j$ (zero for $j > t$).

Output: $\boldsymbol{o}_t = \sum_{j \le t} \alpha_{t,j} \, \boldsymbol{v}_j$.

Key advantage for learning: all T positions are computed in one forward pass, and all T cross-entropy losses are computed simultaneously. Gradients are backpropagated in parallel — no sequential dependency.

Transformer as an AR model

Backprop through self-attention:
For a single head, the attention output $\boldsymbol{o}_t = \sum_j \alpha_{t,j} \boldsymbol{v}_j$ depends on:

- $\boldsymbol{Q} = \boldsymbol{X}\boldsymbol{W}^Q$, $\boldsymbol{K} = \boldsymbol{X}\boldsymbol{W}^K$, $\boldsymbol{V} = \boldsymbol{X}\boldsymbol{W}^V$
- Softmax attention weights $\boldsymbol{\alpha}_t$

Gradient flows back through the softmax and then into $\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}$, and finally into $\boldsymbol{X}$ (from all query and key/value paths). Since $\boldsymbol{X}$ is shared across all heads and projected into $\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}$, gradients accumulate from multiple sources.

Transformer as an AR model

Residual connections and gradient highways:
Each Transformer block has the structure:

$$\boldsymbol{X}' = \boldsymbol{X} + \mathrm{Attn}(\mathrm{LN}(\boldsymbol{X}))$$

$$\boldsymbol{X}'' = \boldsymbol{X}' + \mathrm{FFN}(\mathrm{LN}(\boldsymbol{X}'))$$

By the chain rule, the gradient through a residual block is:

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{X}} = \frac{\partial \mathcal{L}}{\partial \boldsymbol{X}'} \cdot \left(\boldsymbol{I} + \frac{\partial\, \mathrm{Attn}(\mathrm{LN}(\boldsymbol{X}))}{\partial \boldsymbol{X}} \right)$$

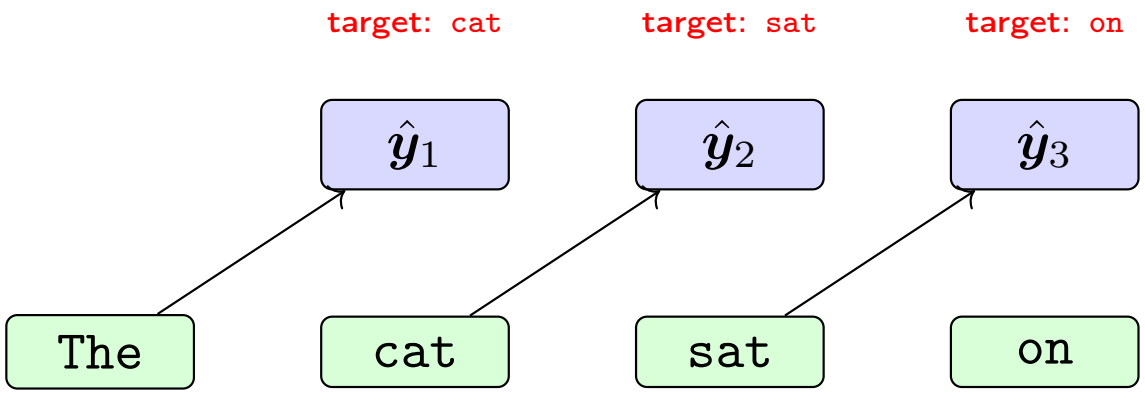
The identity term $\boldsymbol{I}$ ensures the gradient always has a direct path, regardless of how deep the network is. This is why Transformers with 96+ layers can be trained without gradient vanishing.

Transformer as an AR model

Compare and notice that: RNNs had to squeeze gradients through the $\tanh$ bottleneck at every time step. Transformers avoid this entirely.

Teacher forcing

During training, at each position t we feed the *ground-truth* previous token w_t as input, regardless of what the model would have predicted.



Teacher forcing

Why teacher forcing?

- Enables fully parallel training (for Transformers): all targets are known in advance.
- Stabilises gradients: the input distribution is always the true data, not a potentially wrong model prediction.

Drawback (exposure bias): at inference the model must condition on its own predictions, which may differ from the training distribution.

The training loop

Algorithm 1 AR model training (one epoch)

```
for each mini-batch of token sequences do
    Tokenise and pad sequences
    Forward: embed tokens, run encoder, compute logits Z
    Softmax: Ŷ = softmax(Z)
    Loss: L = −(1/T) Σ_t log ŷ_{t,w_{t+1}}
    Backward: ∇_θ L via backpropagation
    Clip: if ‖∇_θ L‖ > τ, rescale
    Update: θ ← θ − η_t · AdamW(∇_θ L)
end for
```

The training loop

Optimiser: AdamW

Adam (Kingma & Ba, 2015) maintains per-parameter first and second moment estimates:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

$$\theta_t = \theta_{t-1} - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t$$

AdamW (Loshchilov & Hutter, 2019) decouples weight decay from the gradient update:

$$\theta_t = (1-\eta\lambda)\theta_{t-1} - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t$$

Default: $\beta_1 = 0.9$, $\beta_2 = 0.95$, $\epsilon = 10^{-8}$, $\lambda = 0.1$ for LLMs.

The training loop

Learning rate schedule:
Flat learning rate leads to instability early in training. Standard practice:

1. **Linear warm-up:** increase η linearly for the first T_w steps (e.g., 2000 steps).
2. **Cosine decay:** decay η following a cosine curve for the remaining steps:

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos \frac{\pi(t - T_w)}{T_{\max} - T_w}\right)$$

The training loop

Gradient clipping: before the update step, rescale gradients if their global norm exceeds threshold τ:

$$\hat{g} = g \cdot \min\left(1,\ \frac{\tau}{\|g\|}\right)$$

Prevents exploding gradients, especially early in training.

Tokenisation

Before a model can process text, tokens must be mapped to integers (and back). This is fixed before training.

Byte-pair encoding (BPE) (Sennrich et al., 2016):

1. Start with a character-level vocabulary.
2. Iteratively merge the most frequent adjacent pair of symbols.
3. Stop after V merges.

Common morphemes (`ing`, `tion`, `un`) and frequent words emerge as single tokens.

Tokenisation

Vocabulary sizes: GPT-2: 50,257 tokens. LLaMA-3 / Qwen-2.5: 128,000+.

Tokenisation directly *affects* gradient computation: a sequence of T tokens yields $T - 1$ training examples in one forward pass, and the embedding lookup gradient is a sparse sum over all occurrences.

Generating text at inference

At inference, we autoregressively sample one token at a time (no backprop):

$$w_{t+1} \sim P_\theta(\cdot \mid w_{\leq t})$$

Common strategies:

- **Greedy:** $w_{t+1} = \arg\max P_\theta$. Deterministic, often repetitive.
- **Top-k:** sample from the top-k tokens; discard the rest.
- **Nucleus (top-p):** sample from the smallest set with cumulative probability $\geq p$.
- **Temperature τ:** divide logits by τ before softmax. $\tau \to 0$: greedy. $\tau > 1$: flatter distribution.

Note: these are *inference-time* choices; they do not affect how the model was trained (which always uses teacher forcing + cross-entropy).

Scaling laws for language models

Kaplan et al., 2020: test loss follows a power-law in parameters N, tokens D, and compute C:

$$L(N) \approx \left(\frac{N_c}{N}\right)^{\alpha_N}, \quad \alpha_N \approx 0.076$$

Chinchilla (Hoffmann et al., 2022): for fixed compute $C \approx 6ND$, scale N and D equally:

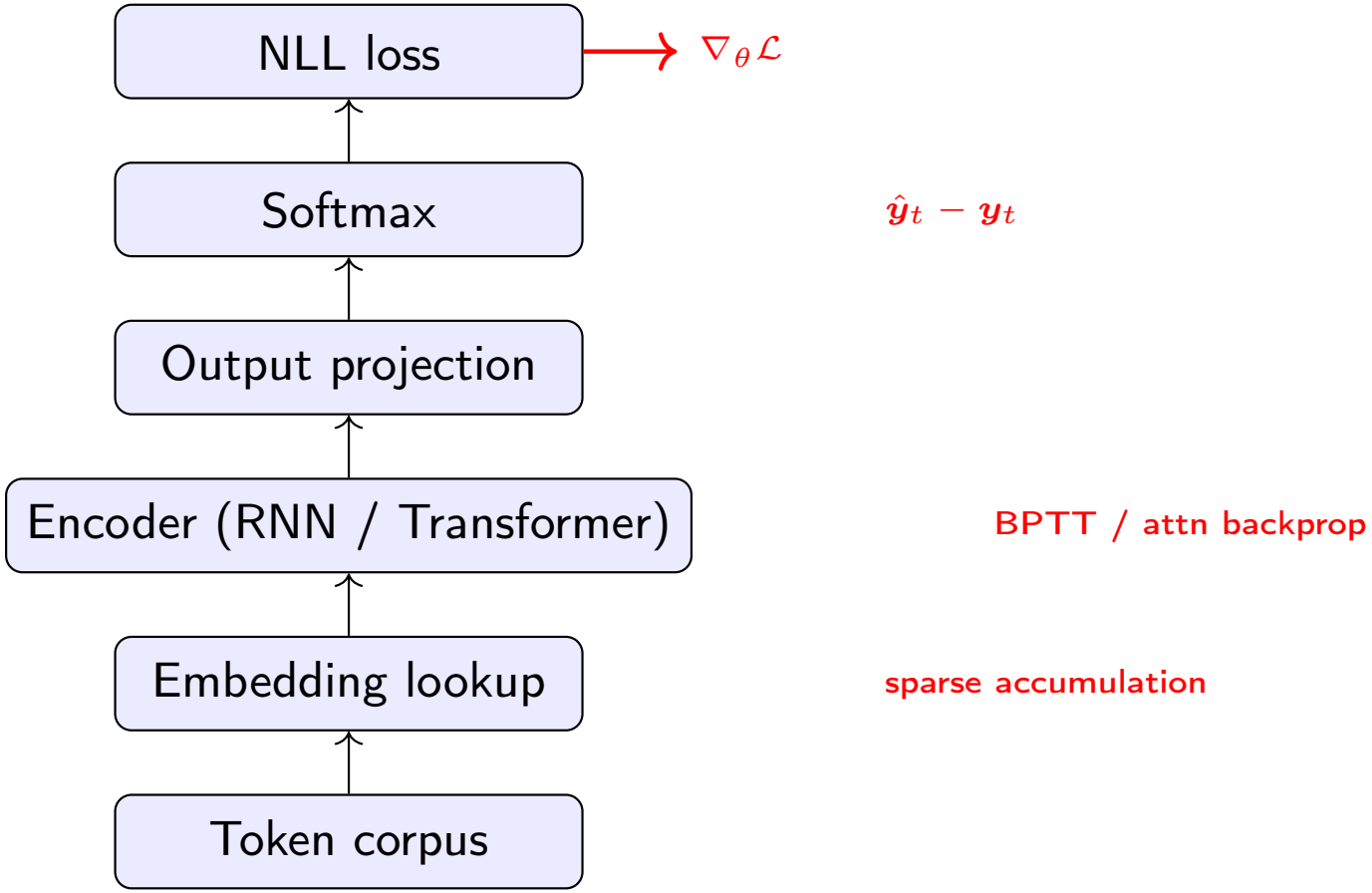
$$N_{\text{opt}} \propto C^{0.5}, \quad D_{\text{opt}} \propto C^{0.5}$$

Scaling laws for language models

GPT-3 (175B params, 300B tokens) was undertrained. Chinchilla (70B params, 1.4T tokens) matched GPT-3 at $4\times$ lower cost.

From a learning perspective: more data means more gradient updates, providing better signal to backprop; more parameters means a larger model can absorb those gradients to encode more knowledge.

Summary: how AR models are learned



References

[1] Y. Bengio, R. Ducharme, P. Vincent, C. Jauvin. A neural probabilistic language model. *JMLR*, 3:1137–1155, 2003.

[2] S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.

[3] A. Vaswani et al. Attention is all you need. *NeurIPS*, 2017. https://arxiv.org/abs/1706.03762

[4] A. Radford et al. Improving Language Understanding by Generative Pre-Training. 2018. https://openai.com/research/language-unsupervised

[5] R. Sennrich, B. Haddow, A. Birch. Neural machine translation of rare words with subword units. *ACL*, 2016. https://arxiv.org/abs/1508.07909

[6] D.P. Kingma and J. Ba. Adam: A method for stochastic optimisation. *ICLR*, 2015. https://arxiv.org/abs/1412.6980

References

[7] I. Loshchilov and F. Hutter. Decoupled weight decay regularisation. *ICLR*, 2019. https://arxiv.org/abs/1711.05101

[8] J. Kaplan et al. Scaling laws for neural language models. 2020. https://arxiv.org/abs/2001.08361

[9] J. Hoffmann et al. Training compute-optimal large language models (Chinchilla). 2022. https://arxiv.org/abs/2203.15556

[10] CS F425 DL Lecture 3 & 4 (Backpropagation in MLP): https://github.com/tirtharajdash/CS-F425_Deep-Learning/blob/main/Lectures/

[11] CS F425 DL Lecture 18 (BPTT): https://github.com/tirtharajdash/CS-F425_Deep-Learning/blob/main/Lectures/Lecture17.pdf

[12] CS F425 DL Lecture 19 (Vanishing gradients in RNN): https://github.com/tirtharajdash/CS-F425_Deep-Learning/blob/main/Lectures/Lecture18.pdf